

Change Report

Cohort 2 Team 9

Charlie Armstrong

Adam Carey

Matthew Havers

Matthew Kelsey

Daniel Packer

Niall Potts

Sam Savidge

Ke Wu

Change Process

We began the process of inheriting team one's design materials by converting the .pdf files on their website into google docs for editing, and cloning their implementation code. We reviewed their assessment materials, discussing areas needing improvement, and areas which needed to change specifically for our team (e.g. the people assigned to scrum roles in method selection and planning).

After this discussion, we noted down the areas which needed to be changed in a google document, and began work on assessment two. Before our team made any changes to the game, a new private repository was created on GitHub with the other team's code so that all new changes could be tracked. A new document was created for tracking major changes to the non-implementation deliverables, which was used to inform the basis of our change report.

Requirements

User requirements

The product brief for assessment two had some new features the customer wanted implemented, therefore we made changes to the user requirements. Achievements and a leaderboard displaying the top five player scores now had to be implemented, so we added UR_ACHIEVEMENTS and UR_LEADERBOARD to reflect the new features. Also, the updated product brief required that we implemented at least five positive events, and three negative/hidden events (previously one of each), so we updated the description of UR_EVENTS to reflect this.

New/updated user requirements

ID	Description	Priority
UR_LEADERBOARD	The system shall have a leaderboard, which displays the name and score of the top five scores.	Shall
UR_ACHIEVEMENTS	The system shall have achievements for notable player interactions with the game, and notify the player if they have gained an achievement.	Shall
UR_EVENTS	The system shall have at least five positive and three negative events, and at least three hidden events alongside a counter showing how many of each event the user has encountered.	Shall

Functional Requirements

Due to the two new user requirements, we added four new functional requirements: FR_DATABASE would be required to implement the leaderboard, as it requires the top five

highest scores to be remembered, and previously there was no way for the game to remember this. FR_LEADERBOARD reflects the game having to provide the player with a display for the player's top five scores. FR_ACHIEVEMENTS reflects having to give the player an in-game notification when they have scored an achievement, and FR_ACHIEVEMENT_SCORE is necessary, as the player's score needs to be altered after gaining an achievement.

Due to the updated UR_EVENTS, we had to make some changes to the related functional requirements. Previously, there was one functional requirement for each event type which described how the actual event would play out in-game. This worked in assessment one as there was only one event for each event type, but if we extended this for assessment two, we would have to describe 11 events in the functional requirements table, which would make the functional requirements table excessively long. Therefore, we decided to restrict the three functional requirements for the events to just a description of each event type.

New/updated functional requirements

ID	Description	User Requirements
FR_DATABASE	The system shall have a database to remember the top five game scores.	UR_LEADERBOARD
FR_LEADERBOARD	The system shall have a leaderboard displaying the player's top five scores	UR_LEADERBOARD
FR_ACHIEVEMENTS	The system shall notify the player has been granted an achievement after the player has done something notable in-game.	UR_ACHIEVEMENTS
FR_ACHIEVEMENT_SCORE	Achievements may affect the player's end score.	UR_ACHIEVEMENTS
FR_POSITIVE_EVENT	The system shall present at least three events which buff the player.	UR_EVENTS
FR_NEGATIVE_EVENT	The system shall present at least five events which hinder player progression.	UR_EVENTS
FR_HIDDEN_EVENT	The system shall present at least three hidden events with special player interactions.	UR_EVENTS

Non-functional/constraint requirements

No new non-functional or constraint requirements were added. The new product brief didn't require any changes to the general operation of the game, only new game features needed adding.

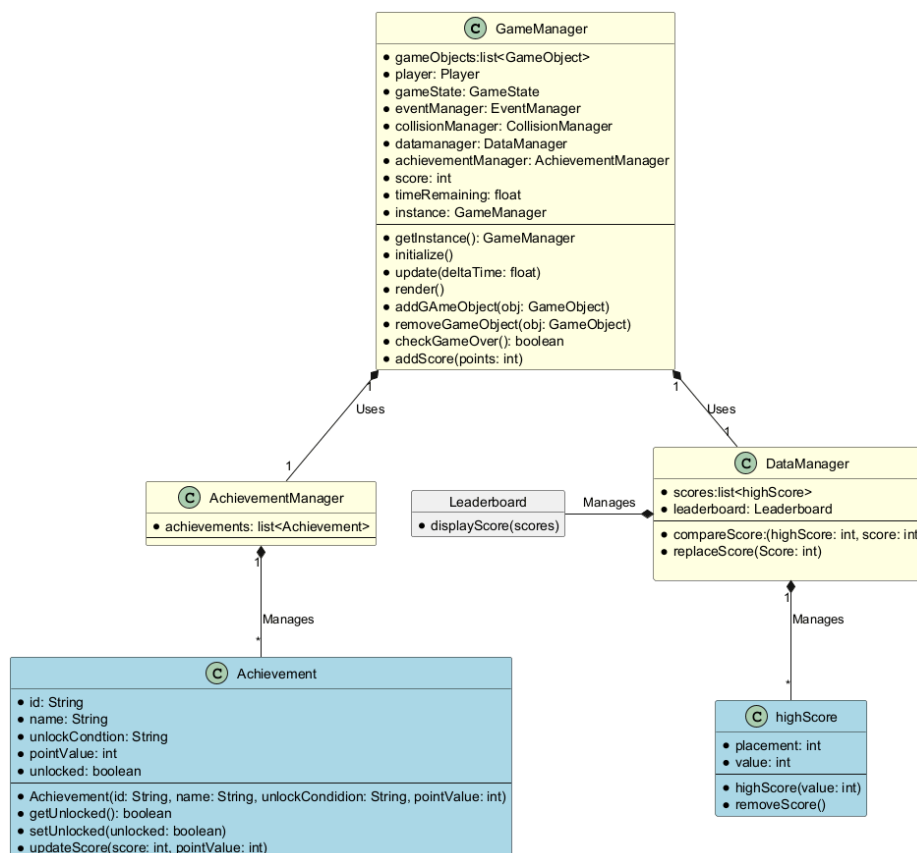
The full requirements table can be found on our [website](#)

Architecture

Structural architecture

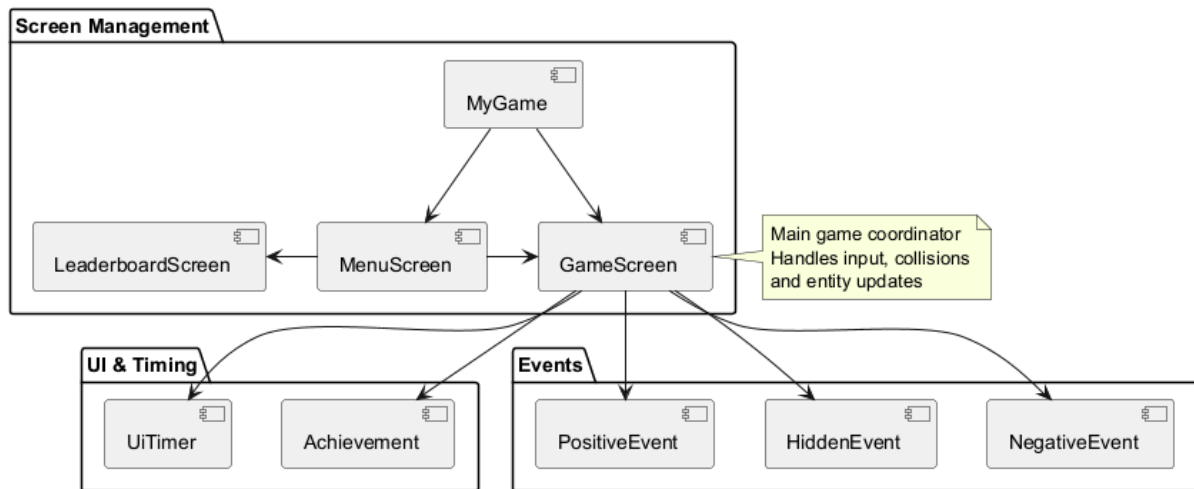
As for OOP component diagram 1, two new classes which use GameManager were added to support the new requirements: AchievementManager uses the Achievement class to store a list holding each achievement a player gained in their current session. DataManager would use the highScore class to store the player's top five scores, compare the last session's score with the current top five scores, and edit the list of scores accordingly.

A snippet of the changes we made to OOP component diagram 1:



As for OOP component diagram 2, we introduced the EquationScreen class to handle an event minigame. It is triggered by an event in GameScreen, after which the game pauses and a new screen is shown, then upon completion the original screen is restored. As for the new leaderboard requirement, we moved the leaderboard logic and player username into the MyGame class; this will act as the central hub to store player data.

A snippet of the changes we made to OOP component diagram 2:



Traceability to requirements

The table in the architecture document showing how each relevant requirements relates to architectural decisions was updated, this can be found on our [website](#)

Method selection and planning

Development method

We kept the previous team's organisation method of agile and scrum. We organised ourselves in the same way in assessment one and this worked for us, so no changes were made to the justification of the development method.

Tools used

In general we used the same tools as the last team in assessment one, so we continued with the same usage of those tools in assessment two. These were:

- GitHub for implementation/testing
- Google Docs for other assessment deliverables
- WhatsApp for communication
- PlantUML/LucidChart for UML/Gantt chart
- Jekyll for website development

Team organisation

The scrum roles were changed to switch their focus to assessment two, and each team member was assigned to at least one role according to the deliverable(s) they were working on. These roles were more partitioned to each assessment deliverable than the other teams – the Christmas vacation and assessment period would make frequent meetings more difficult, so we decided to work more independently for this assessment.

Systematic plan

The table for the project plan was updated for assessment two, and an image of the Gantt chart for the project timeline was added. They can be found [here](#)

Risk assessment

Inheriting another team's project introduces a range of new risks, so we updated the risk table to show this. Three new technical risks (TR7-9) were added to reflect the process of inheriting the previous team's code and documents – the former could have bugs or unimplemented features, and either could be difficult to understand. Time would be lost in correcting these issues and they could lead to bugs in the final product, therefore they are a risk to our project.

Updates were made to the QRR3 risk to reflect the new assessments' increased focus on testing. The tests need to have full scope over the requirements, so there is a risk of missing tests for certain requirements, which might cause unimplemented or failing features to go unnoticed.

A new *participant risk* category was added for the user evaluation. There are several important factors to participants who take part in our study, including how their data is handled, whether consent was properly obtained and their wellbeing during the study. Risks PR1-3 were added to reflect this.

As for the existing risks, the only edits we had to make were for the risk owners. The previous team used their scrum roles for the risk owners and since we changed the scrum roles for the new deliverables (found in the method selection & planning document), we had to make some edits to the risk owners.

The risks which are new or had major changes are found below:

Risk ID	Risk	Likelihood	Impact and Severity	Mitigation	Ownership
TR7	Previous teams code is poorly documented	Possible	2 - Time may be lost understanding documentation, and some code may need to be rewritten or refactored	Before the team starts to write new code, focus on understanding the other team's documentation; rewrite confusing documentation and refactor/rewrite code only if necessary	Developers/architects are to understand the documentation and make any changes to the implementation if necessary
TR8	Previous teams code has bugs/unimple	Possible	4 - Time may be lost debugging and implementing features. If bugs can't be fixed, it would	The previous team's game should be tested briefly before assessment two to get an idea of any	Developers/architects fix bugs and implement new features in the codebase

	mented features		diminish the quality of our product	bugs which need to be fixed. Check the previous teams implementation documents for any features which weren't implemented in assessment one	
TR9	Difficulty inheriting previous team's non-implementation deliverables	Possible	2 - Time may be lost understanding and rewriting deliverables	Review the non-implementation deliverables at the start of assessment two. Identify and document areas which need to be changed on the changelog	Product managers review and document the areas which need to be rewritten. This should be justified in the change report
QRR3	Tests do not adequately cover customer requirements	Possible	4 - Could lead to undetected bugs or unstable features. This will reduce the quality of our game	Tests should be aligned to cover all the customer requirements	Testers are responsible for making the tests, and ensuring they cover all the customer requirements.
PR1	Consent not properly obtained from participants	Unlikely	5 - Harm to participants and consequences from regulatory boards.	Ensure participants sign all the necessary forms before beginning the user evaluation. Store the forms securely for the retention period after the study	The user evaluation team is responsible for obtaining and storing forms signed by participants
PR2	Mishandling of participant data	Unlikely	5 - Harm to participants and consequences from regulatory boards	Ensure participant data is stored securely so that only team members can access. Only the gender/age of participants should be published to avoid de-anonymisation	The user evaluation team is responsible for handling participant data properly and preventing de-anonymisation of participants
PR3	Harm to participants during user evaluation study	Unlikely	4 - Harm to participants	Treat participants with respect and allow them to leave the study if they wish	Any team member who is present during a user evaluation

The full risk table can be found on our [website](#)